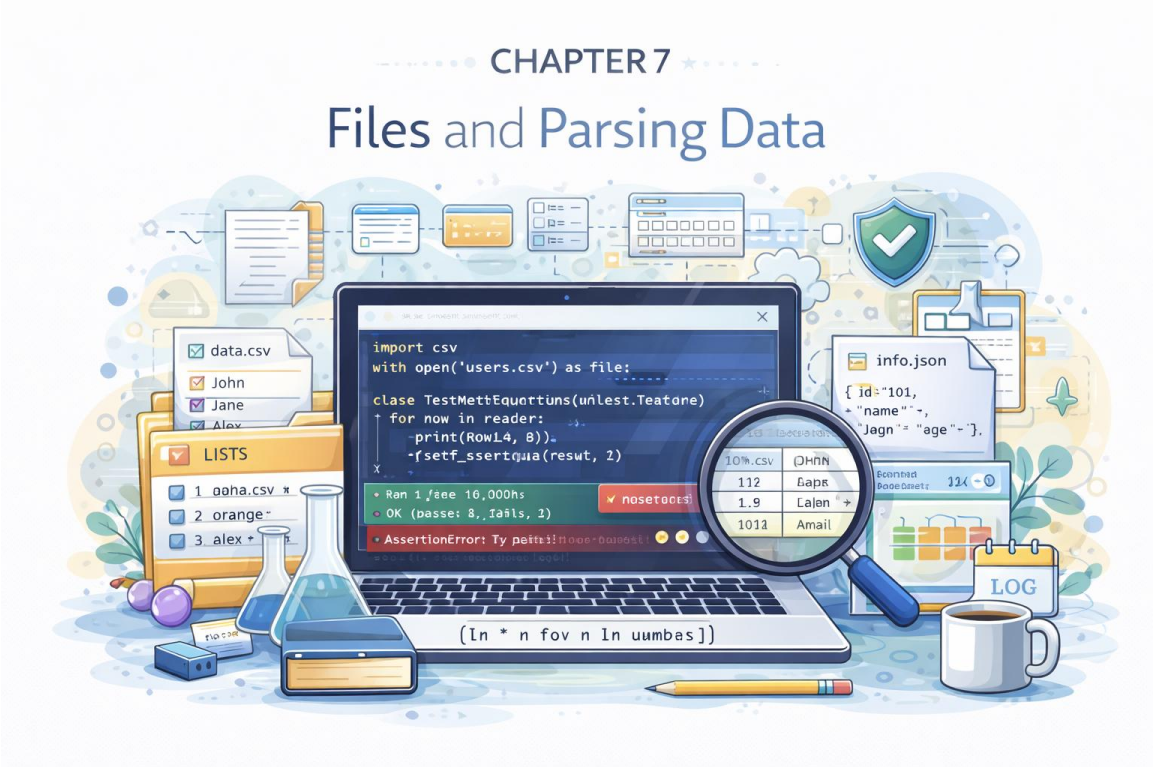

The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 7 – Files and Parsing Data

Programs become truly useful when they can persist data beyond a single execution. Storing information in memory works only while a program is running; once it stops, everything is lost. Files allow programs to save state, exchange data, and integrate with other systems. Nearly every real-world application—whether desktop, web, or mobile—reads from and writes to files in some form.

In Python, file handling is not just about opening a file and reading lines. It involves understanding how data is structured, how it should be interpreted, and how it should be transformed into meaningful program objects. A poorly structured file leads to fragile parsing logic, while a well-structured file allows programs to evolve safely over time.

In this chapter, you will revisit basic file I/O, learn how raw text is parsed, and then move into structured file formats—XML, CSV, and JSON—that Python supports through dedicated libraries. These formats exist specifically to reduce ambiguity, improve readability, and make data easier to share between systems.

7.1 Working with Plain Text Files

Plain text files are the simplest way to store data. They contain characters and line breaks—nothing more. Because of this simplicity, all meaning must be imposed by the programmer. Understanding how and why you parse text files is critical before moving on to more advanced formats.

7.1.1 Reading Files Safely with 'with'

Python provides several ways to open files, but the recommended approach is the `with` statement. It ensures that the file is properly closed, even if an error occurs during processing.

```
with open("inventory.txt", "r") as file:
    for line in file:
        print(line.rstrip("\n"))
```

Key ideas:

- Each line is read as a string
- Line breaks (`\n`) are included unless removed
- `.strip()` removes unwanted whitespace – or casting to `int()/float()` will remove
- The file automatically closes when the block ends

This pattern prevents file corruption and resource leaks and should be used consistently.

7.1.2 Writing Files and Understanding File Modes

Writing to a file follows the same structure but uses a different mode:

```
with open("report.txt", "w") as file:
    file.write("Inventory Report\n")
    file.write("---------------\n")
```

Common modes:

- `"r"` – read
- `"w"` – write (overwrites existing content)
- `"a"` – append

Choosing the correct mode is essential. Accidentally using "w" can erase existing data.

7.1.3 Parsing Text Files Using Known Layouts

Most text parsing relies on one assumption: the programmer knows the file layout. We typically refer to each line in a text file as a record as the data is separated via a delimiter.

Example file (`inventory.txt`):

```
Vacuum Bags    12.99    3
Filter Pack    9.75    8
Extension Hose    24.50    2
```

Each line contains:

- Product name
- Price
- Quantity

```
with open("inventory.txt", "r") as file:
    for line in file:
        parts = line.rstrip("\n").split("\t")
        name = parts[0]
        price = float(parts[1])
        quantity = int(parts[2])
```

This approach works only if:

- The delimiter never changes
- Every line follows the same pattern
- Values never contain commas themselves

As files grow more complex, this approach becomes fragile.

7.1.4 Why Manual Parsing Has Limits

Plain text parsing places all responsibility on the programmer:

- You must validate structure
- You must handle missing or malformed data
- You must ensure consistency across files

Regular expressions can help in some cases, but they add complexity and reduce readability. For this reason, most modern systems rely on structured file formats, which encode meaning directly into the file itself.

Note: Regular expressions are often used to clean or validate text, but they are not covered in this book.

This limitation naturally leads into structured formats like XML, CSV, and JSON.

7.2 CSV Files – Structured Tabular Data

CSV stands for Comma-Separated Values, and it is one of the most common file formats used to store table-style data. A CSV file is essentially a spreadsheet saved as plain text: each line represents a row, and commas separate columns.

CSV is popular because it is:

- easy to create and edit (Excel, Google Sheets, Notepad)
- easy to share between systems
- lightweight and human-readable

However, CSV is still "just text." That means Python reads everything in as strings, and the programmer must decide how to convert values into proper types (numbers, booleans, etc.). CSV is structured, but it is not self-describing of the data contained. Again you are the programmer so most likely you know the structure of the file and what headings will be.

7.2.1 CSV Example File – Fruit Bag Inventory

Here is an example of a CSV file format. Here we have some fruit items in our system:

- name is the item name
- price is the price of item
- quantity is how many items are in stock
- uos is "units of sale" (how many fruit items are inside each bag)

Example file: `inventory.csv`

```
name,price,quantity,uos
Apple,5.99,10,6
Orange,6.49,8,5
Banana,4.25,12,7
```

Notice the first row contains the headers. Headers are critical because they give meaning to each column.

7.2.2 Reading CSV with csv.reader()

Python's `csv.reader()` reads each row as a list of strings. This is fine, but it has a major drawback:

- you must remember column positions (0, 1, 2, 3)
- if the column order changes, your code breaks

```
import csv

with open("inventory.csv", "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

If you use `csv.reader()`, you typically do this:

- manually skip the header row
- convert strings into numbers

```
import csv

with open("inventory.csv", "r", newline="") as file:
    reader = csv.reader(file)

    headers = next(reader)
    print("Headers:", headers)

    for row in reader:
        name = row[0]
        price = float(row[1])
        quantity = int(row[2])
        uos = int(row[3])

        print(name, price, quantity, uos)
```

This works, but it's fragile because it depends on column positions.

7.2.3 Reading CSV with csv.DictReader()

`DictReader` is usually the better tool for real programs because it maps each row into a dictionary, using the headers as keys.

```
import csv

with open("inventory.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    print("Fieldnames (headers):", reader.fieldnames)

    for row in reader:
        print(row)
```

`reader.fieldnames` gives you the headers that `DictReader` detected. This allows your program to discover column names instead of hard-coding them.

7.2.4 Converting DictReader Rows Into Objects

In most applications, you don't want to pass dictionaries everywhere. You want meaningful objects with named attributes and behavior. So we can have an object like so:

```
class Product:
    def __init__(self, name: str, price: float, quantity: int, uos: int):
        self.name = name
        self.price = price
        self.quantity = quantity
        self.uos = uos
```

Then we can use it in the following way:

```
import csv

products = []
with open("inventory.csv", "r", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        p = Product(row["name"], float(row["price"]),
                    int(row["quantity"]), int(row["uos"]))
        products.append(p)
```

At this point, your program has real data structures to work with: a list of Product objects.

7.2.5 Writing CSV with csv.DictWriter()

Remember on our objects we have the dunder method __dict__. By default, this is inherited from our object super class. It automatically takes the name of variables and uses those as the keys and assigns their respective values. This allows us to use the DictWriter when saving back to a CSV file a lot easier.

You may need to define your own__dict__ type method like to_dict() that sets up how you want the object to be written. This may be due to using private/protected variables with underscores in front of the key names. You should NOT override the __dict__ dunder method.

Now that we know we have that dunder method to help, we may need to write data back out. Remember that we want to recreate the same file format with headers and so that both reading and writing workout. This comes when we say add a new object in our list of products here.

```
import csv

with open("inventory.csv", "r", newline="") as file:
    reader = csv.DictReader(file)
    fieldnames = reader.fieldnames
    *** Code here to add into products list ***


with open("inventory_updated.csv", "w", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()

    for p in products:
        writer.writerow(p.__dict__)
```

Reusing `fieldnames` ensures reading and writing stay consistent even if headers change.

This example above is creating a new file, BUT often times you may just overwriting the same file.

7.2.6 Common CSV Pitfalls and Best Practices

- Always use newline="" when opening CSV files.
- Convert numeric values immediately.
- Avoid CSV for nested or complex data.
- Expect dirty or inconsistent real-world data.

CSV is excellent for simple table-based data, but once data becomes hierarchical, formats like XML and JSON are more appropriate.

7.3 XML Files – Structured, Hierarchical Data

XML (eXtensible Markup Language) represents data using nested tags. Unlike plain text files, XML encodes structure directly into the data itself. This makes relationships between pieces of data explicit and easier for programs to understand, validate, and modify.

XML is commonly used when data needs to be shared between systems, stored long-term, or organized in a hierarchical way. Each piece of data lives inside a tag, and related data is grouped together by nesting those tags.

Example XML structure:

```
<inventory>
    <product>
        <name>Apple</name>
        <price>5.99</price>
        <quantity uos="6">10</quantity>
    </product>
    <product>
        <name>Orange</name>
        <price>6.49</price>
        <quantity uos="5">8</quantity>
    </product>
    <product>
        <name>Banana</name>
        <price>4.25</price>
        <quantity uos="7">12</quantity>
    </product>
</inventory>
```

Vocabulary used in XML:

- **Root**: <inventory>
    - Main tag for the entire file
- **Elements**: <product>, <name>, <price>, <quantity>
    - Child tags of the root tags
- **Attributes**: uos="6" (units per bag)
    - Properties you can set on an element tag

7.3.1 Reading XML with ElementTree

Python's `xml.etree.ElementTree` library provides a simple and efficient way to work with XML data. The first step is to parse the XML file and obtain the root element. Remember you as the programmer knows the layout of the file, so you know the Root name and each child element within. You must first get the root of the file.

```
import xml.etree.ElementTree as ET

tree = ET.parse("inventory.xml")
root = tree.getroot()
```

Then using `.findall()` to read through all the child tags within the root:

```
for product in root.findall("product"):
    name = product.find("name").text
    price = float(product.find("price").text)
    quantity_elem = product.find("quantity")
    quantity = int(quantity_elem.text)
    uos = int(quantity_elem.get("uos"))
```

The `.find()` method retrieves a single child element, while `.findall()` retrieves all matching children.

The `.text` property accesses the text inside an element, and `.get()` retrieves attribute values.

7.3.2 Iterating Through XML Data

```
for elem in root.iter():
    print(elem.tag, elem.text)
```

The `iter()` method allows you to walk through every element in the XML tree. This is especially useful for debugging or exploring unfamiliar XML files.

7.3.3 Converting XML to Objects

While XML stores data, programs typically work with objects. Converting XML data into objects allows the rest of your application to operate in an object-oriented way.

Again, referencing our Product object up in 7.2.

```
products = []

for product in root.findall("product"):
    products.append(Product(
        product.find("name").text,
        float(product.find("price").text),
        int(product.find("quantity").text),
        int(product.find("quantity").get("uos"))
    ))
```

7.3.4 Modifying just the XML (No Objects)

If you want to do the most basic modification of XML files like adding a new element you can use the `SubElement()` method. This is for direct modification and not doing parsing to a list of objects.

The `SubElement(parent, tag)` creates a new element and appends it to an existing element inside the parent that you define.

When you complete the modifications you need to `.write()` the changes to the file.

```
import xml.etree.ElementTree as ET
tree = ET.parse("inventory.xml")
root = tree.getroot()

prod = ET.SubElement(root, "product")
name = ET.SubElement(prod, "name")
name.text = "Kiwi"

price = ET.SubElement(prod, "price")
price.text = "5.45"

qty = ET.SubElement(prod, "quantity")
qty.text = "5"
qty.set("uos", "4")

tree.write("inventory.xml")
```

Here we added a new product with the name Kiwi, the price as 5.45, the on hand quantity to 5 and the units of sale to 4 per package. We then write it to the XML file to save the changes.

7.3.5 Writing to XML with Objects

For a different operation in our system as writing our objects out to a file or even creating a new file from scratch there are a couple steps that we need to do.

First we of course will read in the file, save it to a list of products. That has been shown in examples above. But let's say we need to add in another product

```
products.append(Product("Kiwi", 5.45, 5, 4)
```

Then we need to set up our XML file. We first set the root element of the file. Then loop through and add SubElements(), process the root as a tree, and use the tree to write out.

```
root = ET.Element("inventory")
for p in products:

    product = ET.SubElement(root, "product")

    name = ET.SubElement(product, "name")
    name.text = p.name

    price = ET.SubElement(product, "price")
    price.text = str(p.price)

    quantity = ET.SubElement(product, "quantity")
    quantity.text = str(p.quantity)
    quantity.set("uos", str(p.uos))

tree = ET.ElementTree(root)
tree.write("inventory.xml")
```

7.3.6 Formatting XML Text

When we have Python write out our XML file it typically puts it all on one line. While that is OK, it really isn't human readable.

```
<inventory><product><name>Apple</name><price>5.99</price><q
```

The above would be what the format would look like. This is ok for our systems as they can parse through this fine. But if we want to "pretty" it up we can.

For this we need to use the `dom.minidom` module to help us. This module has a `parseString` method that we can send XML string data to and reports back a formatted value to write out. We use the `.tostring()` sending it the root as a function to get the string text format of our XML data.

```
from xml.dom.minidom import parseString
xml_data = ET.tostring(root)

pretty_xml = parseString(xml_data).toprettyxml()

with open("inventory.xml", "w") as file:
    file.write(pretty_xml)
```

There is an interesting nuance with the toprettyxml() function. It likes to add in extra new lines. So our XML file would look like this:

```
<inventory>

    <product>

        <name>Apple</name>

        <price>5.99</price>

        <quantity uos="6">10</quantity>

    </product>
```

It is an interesting nuance that seems like a side effect when the minidom handles text nodes. This is fine. If you can create a new string by using a comprehension to loop through and eliminate the new lines.

```
pretty_xml = "\n".join(line for line in pretty_xml.splitlines() if line.strip())
```

7.4 JSON Files – Structured Data for Modern Applications

JSON stands for JavaScript Object Notation, but despite its name, JSON is not limited to JavaScript. It has become one of the most widely used data formats in modern software development, especially for web applications, APIs, configuration files, and data exchange between systems.

JSON is popular because it is:

- human-readable
- lightweight
- language-independent
- directly compatible with common programming data structures

In Python, JSON maps very naturally to dictionaries and lists, which makes it easier to work with than many other file formats.

Conceptually:

- JSON objects map to Python dictionaries
- JSON arrays map to Python lists
- JSON strings, numbers, and booleans map directly to Python types

This natural alignment is one of the reasons JSON is so heavily used in real-world Python applications.

7.4.1 The JSON Format

JSON represents data using key–value pairs and nested structures. Keys are always strings, while values can be strings, numbers, booleans, lists, or other JSON objects.

Example JSON file using the fruit inventory:

```
[
    {
        "name": "Apple",
        "price": 5.99,
        "quantity": 10,
        "uos": 6
    },
    {
        "name": "Orange",
        "price": 6.49,
        "quantity": 8,
        "uos": 5
    },
    {
        "name": "Banana",
        "price": 4.25,
        "quantity": 12,
        "uos": 7
    }
]
```

Important observations:

- The entire file is a list of objects
- Each object represents one product
- Keys describe the meaning of each value
- Unlike CSV, JSON does not rely on column positions

Because structure is built into the format, JSON is much safer and more flexible than CSV for complex or evolving data.

7.4.2 Loading JSON Data in Python

Python provides the built-in json module to work with JSON files.

```
import json

with open("inventory.json", "r") as file:
    data = json.load(file)
```

After loading:

- data is a Python list
- each item inside the list is a dictionary

For example, `data[0]` might look like:

```
{
    "name": "Apple",
    "price": 5.99,
    "quantity": 10,
    "uos": 6
}
```

At this stage, the data is structured, but it is still just dictionaries. Most applications prefer to work with objects rather than raw dictionaries.

7.4.3 Converting JSON Data Into Objects (Safely)

A common beginner mistake is to assume that JSON keys will always match a class constructor exactly and attempt to pass the dictionary directly into the constructor. This approach is fragile.

A safer, more explicit approach is to map fields manually.

```
products = []

for item in data:
    name = item.get("name")
    price = float(item.get("price", 0.0))
    quantity = int(item.get("quantity", 0))
    uos = int(item.get("uos", 1))

    product = Product(name, price, quantity, uos)
    products.append(product)
```

Using .get():

- prevents `KeyError`
- allows default values
- makes your code more resilient to data changes

7.4.4 Writing JSON Files with json.dump()

If our data is already in a list of dictionary format we can simply use the `dump()` function to write out our JSON files.

```
with open("inventory.json", "w") as file:
    json.dump(data, file)
```

By default, this produces compact JSON that is difficult to read. Everything is on one line, just like XML. It is a little easier to format here though.

7.4.5 Formatting JSON Output: indent and sort_keys

To make it easier to read we can set the indentation and decide if we would like to sort keys or not.

```
with open("inventory.json", "w") as file:
    json.dump(data, file, indent=4, sort_keys=True)
```

The `indent` property places each key and value on its own line with indentation, making the file human-readable.

The `sort_keys` property writes the dictionary keys in alphabetical order, improving readability.

7.4.6 Writing Objects to JSON Using __dict__ or custom method

If our data isn't in a dictionary format, like a list of objects, then we need to convert our format. We can leverage the `__dict__` method or depending on the attributes of our objects we may need a custom `to_dict()` on our objects. This would be if our attributes/variables were protected or private. For example, if our Product object was like the following, we need a `to_dict()` setting up our specific keys.

```python
class Product:
    def __init__(self, name: str, price: float, quantity: int, uos: int):
        self.__name = name
        self.__price = price
        self.__quantity = quantity
        self.__uos = uos

    def to_dict(self):
        return {"name": self.__name, "price": self.__price,
                "quantity": self.__quantity, "uos": self.__uos}
```

Here if we are to use our custom dictionary method we would do something like the following:

```python
json_data = [p.to_dict() for p in products]

with open("inventory.json", "w") as file:
    json.dump(json_data, file, indent=4)
```

Here we are using the built in `__dict__()` to create a list of all our products (if our Product had no private variables)

```python
json_data = [p.__dict__ for p in products]

with open("inventory.json", "w") as file:
    json.dump(json_data, file, indent=4)
```

7.4.7 When JSON Is the Right Choice

JSON is an excellent choice when:

- data needs to be shared between systems
- structure may evolve over time
- nested data is required
- human readability matters
- APIs are involved

Chapter 7 – Key Takeaways

- Files allow programs to persist data beyond execution and serve as interfaces between systems, users, and applications.
- Plain text files are simple and human-readable but place all responsibility for structure and validation on the programmer.
- CSV files are best suited for flat, tabular data and business reporting, especially when data needs to be viewed or edited in spreadsheet software.
- XML files represent hierarchical data with strict structure, making them suitable for enterprise systems, configuration files, and regulated environments.
- JSON files balance structure, readability, and flexibility, making them the dominant format for modern applications and APIs.
- Python provides dedicated libraries (csv, xml.etree.ElementTree, and json) that simplify parsing, modifying, and writing structured data.
- Converting file data into objects allows applications to operate in an object-oriented way rather than manipulating raw dictionaries or strings.
- The __dict__ attribute provides a natural bridge between objects and file formats like CSV and JSON, but custom to_dict() methods offer greater control when needed.
- Choosing a file format is both a technical and business decision, influenced by data structure, longevity, integration requirements, and who will consume the data.
- The best file format is the simplest one that still enforces the business rules and data integrity your system requires.